

scFoundation: learning cell states at transcriptome scale

Journal Club | Biological Foundation Models

Reading edition: 7 September 2026

1. Title

This commentary concerns the **2024 version of record**, published in *Nature Methods* 21, 1481–1491, rather than an earlier preprint. The model is also named **xTrimoscFoundation α** . The central reading question is whether a pretrained expression representation can become a reusable component across biological tasks, and what further evidence would be needed before using that component in a virtual embryo. Paper

The numerical plots below were newly drawn from the publisher-hosted Supplementary Information; they are not copied paper figures or independently reproduced experiments. Implementation observations refer to the authors' repository at commit 397631c495eddf9ad6644fc00c6ea8139e651245, inspected on 7 September 2026. No checkpoint was run for this commentary. Supplement · Code snapshot

Resources: published article · Supplementary Information · processed data and embeddings · archived code · xTrimoGene architecture paper. Dataset accessions and collection manifests should be taken from the original article's Supplementary Data 1–2; this commentary does not substitute for a study-by-study overlap audit.

2. Abstract

scFoundation asks whether large-scale expression pretraining can supply useful representations for multiple downstream analyses. Its training corpus contains more than 50 million human single-cell profiles, aligned to 19,264 genes, and its largest model has approximately 100 million parameters. The architecture combines an encoder that processes observed, unmasked genes with a lighter decoder that restores the complete gene representation. Read-depth-aware pretraining asks the network to reconstruct expression while distinguishing the observed sequencing depth from a requested target depth. Paper

The evidence is best read as a collection of task-specific tests rather than one universal capability score. Annotation, drug-response modeling and perturbation modeling use different downstream heads and training regimes. The results support testing scFoundation as a reusable cell-state or gene-context encoder, but they do not demonstrate a standalone simulator of cell development. For virtual-embryo research, the useful distinction is between representing the state of a cell and learning how that state changes with developmental time, perturbation and spatial context. This commentary therefore focuses on what is transferred, how success is measured, and which assumptions would need new validation.

3. Main

3.1 The scientific problem and the study's logic

A transcriptome vector contains both biological information and properties of its measurement. Two cells can appear different because they occupy different states, because they were sampled at different depths, or because both changed. A transferable representation should preserve biologically useful differences without mistaking every technical difference for a new state. Conversely, removing variation indiscriminately risks erasing the developmental processes that a later analysis is trying to study. This is the interpretive problem against which I read scFoundation.

The study's logic is to learn a general representation first and then test its usefulness through several distinct downstream tasks. The released code makes this modularity explicit: cell embeddings are exported to DeepCDR and SCAD, whereas cell-specific gene embeddings are supplied to GEARS. These are different interfaces, not interchangeable names for the same prediction. DeepCDR implementation · SCAD implementation · GEARS implementation

3.2 Data scope and units

Pretraining draws on human scRNA-seq collections including GEO, HCA, Single Cell Portal, EMBL-EBI, hECA and DISCO. The corpus spans more than 100 tissue types and includes normal, disease and tumor material. It is not a staged embryo training set. The reported pretraining validation set contains 100,000 randomly sampled cells. Paper

The downstream data serve different purposes: Baron pancreatic data test depth enhancement; Zheng68K blood cells and Segerstolpe pancreatic cells test annotation; cancer cell-line data support drug-response modeling; and Adamson, Dixit and Norman Perturb-seq data support perturbation prediction. The supplementary perturbation counts are **87 single-gene conditions, 24 single-gene conditions, and 105 single-gene plus 131 two-gene conditions**, respectively. These counts describe interventions, not numbers of cells. Processed inputs are linked through the authors' data release. Supplement, Note 5 · Processed data

For reuse, I would separately record the study, donor, protocol, biological condition and number of cells retained after preprocessing. A large corpus size does not establish uniform developmental coverage. The exact post-filtering cell count in each downstream split and the full pretraining overlap manifest were not independently audited here.

3.3 What enters and leaves the model

Interface	Input	Output and intended use
Cell representation	One expression profile in the fixed gene order, with depth indicators	A pooled embedding for clustering or a separately trained prediction head
Gene representation	The same profile, passed through encoder and decoder	A context vector for each of 19,264 genes; suitable as gene-graph node features
Expression reconstruction	A masked or lower-depth profile with source/target depth information	Predicted expression values, not new experimental observations
Task-specific prediction	Exported embeddings plus task inputs such as a drug or perturbation	The downstream model's output, not an intrinsic output of the pretrained encoder alone

The inference script separates cell, gene, gene_batch and gene_expression outputs, and explicitly marks direct expression output as not recommended. Consequently, I would start by testing embeddings rather than treating reconstructed counts as replacement measurements. Inference code

4. Methods Highlights

4.1 Representation: gene identity plus expression value

The model requires a fixed ordered gene list, not a sequence ranked by expression. Its input combines a learned gene-identity embedding with a learned value embedding. An equivalent schematic notation is

$$h_i = e_{\text{gene}}(g_i) + e_{\text{value}}(x_i),$$

where g_i identifies gene i , x_i is its processed expression value, and h_i is the vector supplied to the network. This equation summarizes the implementation; it is not a new biological model. Gene identity answers *which gene?*, while the value embedding answers *how much expression?* Inference code

The xTrimoGene companion paper describes a learnable mapping of continuous expression into vectors rather than fixed rounding into expression bins. It also distinguishes zero and masked positions. That distinction matters: a deliberately withheld value and an observed zero have different meanings in the reconstruction task. It does not, by itself, distinguish technical dropout from true absence of transcription. xTrimoGene

4.2 Why the encoder and decoder are asymmetric

The computational idea is to reserve expensive full attention for the shorter set of nonzero, unmasked genes. The decoder then receives the encoded tokens together with zero/mask representations, allowing predictions over the full gene list. The encoder uses standard transformer attention; the lighter decoder uses Performer-style approximate attention. This is the architectural contribution of xTrimoGene that makes long transcriptome inputs more tractable. xTrimoGene, Fig. 1

For the 100-million-parameter configuration, the encoder has 12 layers with 768-dimensional states, while the decoder has 6 layers with 512-dimensional states. Concatenating four encoder summaries gives a 3,072-dimensional cell vector; decoder output provides 512 features per gene. These dimensions matter when choosing a downstream head or budgeting storage. Supplementary Table 7 · Inference code

This separation has a biological consequence worth testing: an observed gene receives a context-dependent representation rather than one fixed vector used in every cell. Two cells can therefore assign different contextual roles to the same gene. Nevertheless, a context-dependent vector is a statistical representation; naming it a regulatory interaction requires additional evidence.

In the released inference code, cell pooling concatenates the source-depth token, target-depth token, maximum and mean over encoded gene tokens when `pool_type=all`. Gene output instead removes the final scalar projection and retains the decoder's per-gene vectors. The default is therefore not a generic language-model CLS token. The pooling choice should be recorded whenever embeddings are compared. Inference code

4.3 Read-depth-aware learning: depth is not developmental time

The pretraining task combines optional count downsampling, masking and squared-error reconstruction. The source-depth indicator S describes the input; T describes the desired target depth. The masking ratio is 30%. A compact restatement of the objective is

$$\mathcal{L}_{\text{RDA}} = \frac{1}{|M|} \sum_{i \in M} (\hat{x}_i - x_i)^2,$$

where M is the masked gene set, x_i is the normalized, log-transformed target, and $\hat{x}_i$ is the model prediction. This is a regression objective, not a negative-binomial count likelihood or a VAE evidence lower bound. Paper, Methods · xTrimoGene

The important conceptual point is that changing T asks for a different measurement-depth condition. It does **not** ask the cell to advance to a later developmental stage. The task encourages robustness to a chosen corruption process; whether that process represents the experimental noise in a new embryo dataset is an empirical question.

4.4 An implementation detail that changes the biological interpretation

For raw single-cell input, the inspected code normalizes each cell to a total of 10,000 and applies `log1p`. It then appends logarithmic depth indicators. With a source total S , `--tgthighres f2` encodes a target total of $2S$; `t4` specifies a target log-depth of 4, corresponding to 10,000. The `a` option adds to log-depth, not to raw molecule counts. These semantics should be checked before interpreting an enhancement experiment. Inference code

The `pre_normalized=A` route accepts normalized expression with the original total appended. This is useful when raw counts have been retained separately: the sum of already log-normalized values should not silently be treated as the original molecule total. The script also only automatically invokes its gene-selection helper under certain input-width conditions. I would therefore explicitly align names and order even when a matrix already has 19,264 columns. A matching shape is not evidence of a matching gene order. Inference code · Model documentation

4.5 What is actually trained in each benchmark?

Experiment	Role of scFoundation	Additional learning or evaluation
Read-depth enhancement	Pretrained representation without dataset-specific fine-tuning	Clustering and comparison with reference labels
Cell-type annotation	One encoder layer is fine-tuned	MLP classifier; random 8:1:1 cell splits
Bulk drug response	Exported cell-line embeddings	Train DeepCDR with drug-structure features
Single-cell drug response	Exported cell/bulk embeddings	Train SCAD domain adaptation; fivefold evaluation
Perturbation prediction	Frozen gene-context encoder in the reported experiments	Train GEARS; hold out perturbation conditions

The annotation protocol is described in the paper’s Methods; the downstream repositories distinguish frozen feature extraction from training the prediction model. In particular, GEARS documentation permits joint fine-tuning but states that the reported experiments used the frozen option. The existence of a later software option is not evidence that it generated a published result. Paper · DeepCDR · SCAD · GEARS

A random-cell split evaluates a different question from an unseen-donor, unseen-study or unseen-developmental-stage split. Likewise, an unseen pair of perturbed genes is not necessarily a pair whose individual genes were both absent during training. These distinctions should remain visible when comparing the results with a future virtual-embryo benchmark.

4.6 Reproducibility and practical reuse

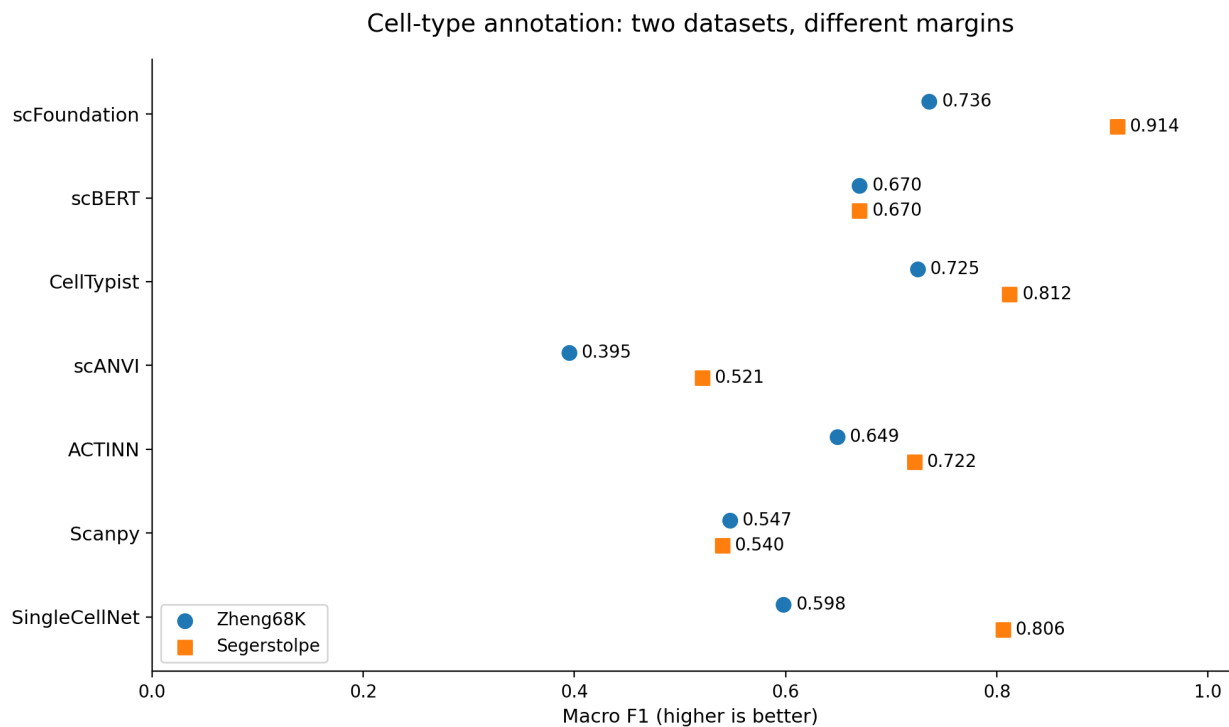
The repository supplies example commands, exported embeddings, downstream scripts and links to processed data. However, its components have separate environments: the DeepCDR README lists Keras 2.1.4 and TensorFlow 1.13.1, while local foundation-model inference uses PyTorch. I would isolate these environments rather than assume a single modern installation reproduces the entire paper. Neither runtime nor peak memory has been measured in this reading. DeepCDR · Model documentation

A first replication should freeze the code snapshot, checkpoint, gene list, normalization, depth setting, pooling, split and random seed. Start with a small embedding export and verify output dimensions before running a task-specific head. The checked inference script contains explicit CUDA calls, so an advertised CPU fallback should not be inferred from the presence of a device variable. Inference code

5. Results

5.1 Annotation improves, with a dataset-dependent margin

Supplementary Table 4 reports macro F1 of **0.736 versus 0.725** for scFoundation and CellTypist on Zheng68K, and **0.914 versus 0.812** on Segerstolpe. The absolute gains are 0.011 and 0.102. These are supervised annotation results, not zero-shot embryo-stage prediction. Supplementary Table 4



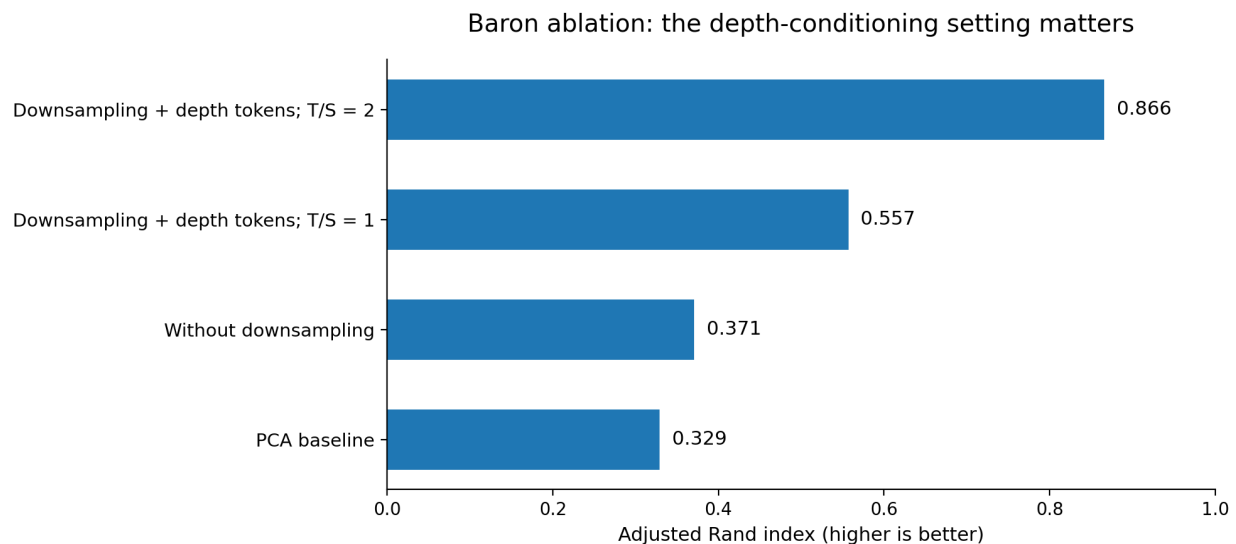
Source: Hao et al. (2024), Supplementary Table 4. New plot of reported values; no independent rerun or inferred error bars.

Journal Club Figure 1. Newly plotted values from Hao et al., Supplementary Table 4. Each point is an author-reported macro F1; higher is better. The table does not provide confidence intervals, so none are drawn. This plot is not a rerun of the benchmark.

The two gains should not be compressed into one generic improvement claim. A small aggregate advantage can still be useful, but it calls for uncertainty estimates and per-class inspection. For deployment I would inspect rare-state recall, confusion between adjacent developmental states and sensitivity to annotation granularity. A visually cleaner embedding would not replace those checks.

5.2 Depth conditioning is a substantive design choice

In the Baron ablation, ARI is **0.866** with downsampling and $T/S = 2$, **0.557** at $T/S = 1$, **0.371** without downsampling, and **0.329** for the PCA baseline. The ablation also changes the availability of depth tokens, so it should not be interpreted as isolating downsampling alone. Supplementary Note 1 and Table 8



Source: Hao et al. (2024), Supplementary Table 8. T/S denotes sequencing depth, not developmental time.

Journal Club Figure 2. Newly plotted ARI values from Hao et al., Supplementary Table 8. The T/S ratio refers to target versus source sequencing depth, not time. These are ablation settings and must not be substituted for the separate pooling experiments in Supplementary Table 9. No replicate-level uncertainty is supplied in Table 8.

This result motivates a depth-sensitivity analysis whenever the encoder is reused. I would choose the depth setting on training/validation data, then hold it fixed for the test set. Selecting it using test-cell labels would turn a useful control into test-set tuning. In developmental data, I would additionally check whether depth conditioning changes the apparent abundance or separability of transient populations.

5.3 Drug-response transfer is informative but task-specific

For the experiment illustrated in original Fig. 3, the released DeepCDR README gives expected overall Pearson correlations of **0.8371** for the expression baseline and **0.8783** for the embedding-based model. These are repository-reported expected outputs, not newly computed estimates. The gain is 0.0412 in correlation units, not a percentage reduction in prediction error. DeepCDR expected outputs

The SCAD README provides similarly concrete examples associated with the original Fig. 4 task: AUROC rises from **0.56 to 0.84** for sorafenib and **0.62 to 0.84** for NVP-TAE684, while etoposide changes from **0.66 to 0.68**. The transfer benefit is therefore not uniform across drug settings. Its example baseline and embedding commands also differ in downstream hyperparameters, which makes an equal-tuning-budget comparison a useful replication target. SCAD expected outputs and commands

The mechanism of transfer is worth retaining: a pretrained expression representation can be reused alongside drug information. Clinical efficacy, patient-specific treatment selection and transfer to a new developmental context are separate endpoints. Good prediction in these benchmarks does not supply those additional validations.

5.4 Perturbation prediction tests the encoder-plus-GEARS system

Original Fig. 5 evaluates post-perturbation expression and genetic interactions, comparing the scFoundation-enhanced GEARS pipeline with baselines. The gene vectors enter GEARS as cell-specific features; the reported setup freezes scFoundation and trains the downstream predictor. This is stronger than merely showing a plausible latent-space plot, but it is not zero-shot perturbation simulation by scFoundation alone. Paper, Fig. 5 · GEARS implementation

The relevant evaluation unit is a held-out intervention. Because destructive single-cell assays do not provide a matched before-and-after measurement for each individual cell, I would inspect both average responses and how well the predicted population reflects heterogeneity. Accuracy on selected differentially expressed genes should be accompanied by broader expression and cell-state checks before claiming a complete response model.

5.5 Gene modules support interpretation, not causal identification

The paper uses gene embeddings for module analysis and combines similarity with SCENIC-based information for regulatory exploration (Supplementary Figs. 9–13). This is an interpretable use of context vectors, not independent experimental confirmation that each inferred edge is causal. Paper

For a developmental application, a useful next test would ask whether a candidate module remains informative across held-out stages and whether perturbing a nominated regulator changes the predicted fate in the expected direction. Module enrichment can prioritize such tests; it cannot substitute for them.

6. Novelty

6.1 What is genuinely useful here?

My main takeaway is the combination of three reusable ideas: efficient handling of sparse, long gene inputs; an explicit control for measurement depth; and separate cell-level and gene-level interfaces. The architecture makes the model usable as a component, rather than forcing every downstream question into a single output type. Its contribution is primarily methodological representation learning, not the discovery of one new developmental mechanism.

The distinction between identity and context is particularly useful. A static gene lookup gives the same starting vector wherever a gene appears; the network can turn that into a cell-specific representation by conditioning on the observed transcriptome. The inference code exposes both the pooled cell description and the detailed gene context. That design suggests concrete comparisons between cell-level models and models that preserve gene-level structure. Inference code

6.2 Where I would be cautious

Benchmark interpretation. The scaling panel in original Fig. 2a estimates xTrimoGene models at parameter budgets associated with other architectures; it is not a direct rerun of every named pretrained model. Supplementary Note 2 also describes a different training/input protocol for scVI. These distinctions prevent that panel from establishing a universal ranking of foundation models. Supplementary Note 2

Measurement assumptions. Zero padding makes the input interface convenient, but a gene absent from a panel is not biologically identical to a measured zero. The implementation can run after padding without proving that its representation remains reliable. I would quantify missing-gene coverage and its association with cell types, stages and platforms before interpreting any downstream improvement.

Coverage and leakage. Randomly holding out cells is legitimate for the question it addresses, but it cannot establish independent-donor or independent-study generalization. It also does not by itself settle overlap between a public pretraining collection and a downstream benchmark. This commentary has not demonstrated leakage; it identifies a provenance check required for a stronger transfer claim.

Objective alignment. A reconstruction model is rewarded for predicting expression under its training corruptions. The biologically important signal in a new problem may be a rare transient state rather than the

dominant expression pattern. I would assess preservation of that signal directly, rather than assume that lower reconstruction error guarantees better developmental inference.

6.3 Implication for virtual embryos and cell-state transitions

My proposed use is an encoder, not a replacement for the developmental model. In schematic notation, one could compute $z_i(t) = E_\phi(x_i(t))$, where $x_i(t)$ is an observed transcriptome at developmental time t , and E_ϕ is the pretrained encoder. A separate model would then have to learn temporal change, spatial interactions or perturbation responses. This is a suggested application of the representation, **not an experiment performed in the scFoundation paper**.

The first decisive test would compare frozen scFoundation features with a compact, dataset-trained representation under the same downstream model and the same held-out embryos or stages. That design separates an encoder advantage from a more powerful prediction head. I would retain the foundation encoder only if it improves the biological endpoint, preserves transitional states and remains stable to sensible depth settings.

Three distinctions guide that test: reconstruction versus future-state prediction; cross-cell generalization versus cross-embryo generalization; and expression similarity versus fate or regulatory mechanism. A failure on one distinction need not invalidate the model, but it should narrow the claim and the role assigned to it.

Overall assessment: scFoundation is a valuable architecture and transfer-learning case study. Its strongest lesson for my reading programme is how to build and evaluate a reusable representation interface. Whether that interface improves a predictive virtual embryo remains a separate, testable question.

Sources and provenance

The links below identify the original research and the implementation resources used in this commentary. The published text was consulted through a publicly readable reproduction and cross-checked against publisher-hosted supplementary material and author code; publisher PDF pages and figures are not redistributed. Repository expected outputs, published table values and my proposed experiments are explicitly distinguished above.

Source	Role in this reading
Hao et al., Nature Methods (2024)	Published study, bibliographic record, task definitions and original figure references
Publisher-hosted Supplementary Information	Numerical Tables 4 and 8; ablations and benchmark qualifications
Published-text access copy	Access to the original article text, excluding the hosting site’s generated synopsis
Gong et al., xTrimoGene, NeurIPS (2023)	Architectural companion; not substituted for scFoundation benchmark results
Author repository snapshot	Implementation context at the inspected commit
Inference source and model guide	Gene ordering, normalization, depth indicators and output interfaces
DeepCDR, SCAD, GEARS	Downstream configurations and explicitly labeled expected outputs
Figshare data release and Zenodo archive	Reproduction resources; not downloaded or re-executed here